Sylvia Ekström October 28, 2014

Quick start manual for ORIGIN_TOOLS
(v. 4.0.3)

1 Reading of files

- **Loading an EVOL file**
  Command: `loadE('#file',num_⋆,num_start=#line,num_end=#line,format='format')`
  Example: `loadE('~/calc/Z002/P015z02S0/P015z02S0.wg',1)`
  `loadE('~/grids/tables/M015z02S0.dat',2,format='tgrids')`
  The existing formats are `o2013,tgrids,tools` (default: `o2013`).

- **Loading a STRUC file**
  Command: `loadS('#file',num_⋆,num_start=#line,format='format')`
  Example: `loadS('P015z02S0.v0001001',1`
  `loadS('P015z02S0_StrucData_0001001.dat',2,format='full')`
  The existing formats are `o2013,full` (default: `o2013`).

NB: switching from EVOL mode to STRUC mode with `switch('struc')` and back with `switch('evol')`

2 Models selection

- **Selection:**
  Command: `select_model(num_⋆)`
  Example: `select_model(2)`
  `select_model([1,2,3,4,5])`

- **Addition of a model:**
  Command: `add_model(num_⋆)`

- **Removal of a model:**
  Command: `del_model(num_⋆)`

- **Recall of loaded models:**
  Command: `LoadedEvol()` (EVOL)
  Command: `LoadedStruc()` (STRUC)

3 Variables

- **List of available variables**
  Command: `VarEvol()` (EVOL)
  Command: `VarStruc()` (STRUC)
  (see also section 6)

- **Get the values of a variable for a model**
  Command: `Get_Var('variable',num_⋆)`

- **Addition of a new variable**
  Command: `Set_Var(newVar,'Var_Name',num_⋆,label='#LaTeX_string',category='category')`
  with the following existing categories (but it's possible to create one) :
  - EVOL : `model,surface,centre,abundances,rotation,winds,energetics`
  - STRUC : `structure,thermo,EOS,energy,abundances,rotation,magnetism`

  Example: `VVc = Get_Var('Vsurf',1)/Get_Var('Vcrit1',1)`
  `Set_Var(VVc,'VVc',1,label='$V/V_\mathrm{crit}$',category='rotation')`

4 Graphics

4.1 Basics

- **Definition of the x axis**
  Command: `defX('variable')`

As default, the axis is t [Myr] in EVOL mode and M_r in STRUCmode

- **Basic command**
  Command: `Plot('variable')`

- **Graph with variable(s) in log**
  Command: `log_Var('axis')`
  with `axis=x,y,xy`, then `Plot('variable')`

- **Ratio between two variables**
  Command: `plotRatio('var_1','var_2')`
  The optional argument `index=#line` allows to divide variable 1 by the value of variable 2 at that line. Usefull for example to plot the relative evolution of a variable with respect to its initial value :
  `plotRatio('var','var',index=0)`.

- **3rd variable in colour code over the curve**
  Command: `Plot_colour('var_y','var_z')`
  The min and max values of `var_z` are determined automatically, but it is possible to modify this behaviour with `CBLimits(min=#min,max=#max)`, each argument being optional. To get back to an automatic determination, the command is `no_CBLimits()`, which accepts the argument `min` or `max` if only one value has to be redetermined.
  The colour map can be changed with the command `set_colourMap('map')` where `'map'` is one of the colour maps proposed by `matplotlib` (see the documentation page)

4.2 Predefined graphs

EVOL:

- **HR diagram**
  Command: `HRD_plot()`

- **Kippenhahn diagram**
  Command: `Kippen(num_⋆)`

- **log T_c − log ρ_c diagram**
  Command: `rhoT()`

- **Y − log T_{eff} diagram**
  Command: `YTeff()`

- **Abundances evolution**
  Command: `Abund('c')` (centre)
  Command: `Abund('s')` (surface)

- **N/C vs N/O diagram**
  Command: `NCNO()`

- **Set of plots summarising a model**
  Command: `Hirschi_plots(num_⋆)`
  This command automatically generates the following plots:
  - window 1: HRD, Abund(c), Kippenhahn, L and T_{eff} vs time
  - window 2 : V_{surf}, Ω/Ω_{crit}, $\dot{M}$, Ω_c/Ω_s
  - window 3 : Abund(s), N/O, N/H, N/C
  - window 4 : $T_c - \rho_c$, T_c and ρ_c vs time

  With the optional argument `'legend'`, a legend is added to the plots.

STRUC:

- **log T − log ρ diagram**
  Command: `rhoT()`

- **Gradients (∇_{ad} - ∇_{rad} - ∇_{μ})**
  Command: `Nablas(num_⋆)`

- **Abundances profile**
  Command: `Abund('p')`

- **addition of the convective zones on the plot**
  Command: `convZones(num_⋆)`

- **Set of plots summarising a structure**
  Command: `Hirschi_plots(num_⋆)`
  This command automatically generates the following plots:
  - window 1 : Abund(p)
  - window 2 : T, r, P, ρ
  - window 3 : L and ϵ, T vs ρ, ϵ_{reac} and ϵ_{grav}, nablas and kappa
  - window 4 : D, N^2, c_{sound} eandt V_{MLT}, magn. field variables
  - window 5 : Ω and $\Omega/\Omega_{\text{crit}}$, V_{eq}, j, U and V

  With the optional argument `'legend'`, a legend is added to the plots.

4.3 Saving the figures

The python window allows to save the current figure in png format, with a clic on .

For a vectorised format, the command `MyFig('#file_name')` can be used. The default format is pdf, but it might be customised with the optional argument `format='#format'`.

5 Adaptations to specific needs

5.1 Axes

- **Change of the limits**
  Command: `Limits(xmin=,xmax=,ymin=,ymax=)`
  NB: back to normal with `noLimits('x','y' ou 'xy')`. `noLimits()` without argument corresponds to `'xy'`.

- **Inversion of an axis**
  Command: `axis_inv('x','y' or 'xy')`
  NB: back to normal with `no_axis_inv('x','y' ou 'xy')`. `no_axis_inv()` without argument corresponds to `'xy'`.

- **Axis in logarithmic scale**
  Command: `logScale('x','y' or 'xy')`
  NB: back to normal with `no_logScale()`

- **Modification of an axis label**
  Command: `change_label('axis','label')`

5.2 Points/curves

- **Switch from curves to points**
  Command: `Points(True)`

- **Choice of a colour sequence when several models are plotted together**
  Command: `set_colourSequence('i' or 'c')`
  `'c'` stands for `'contrast'`: black,red,green,blue,cyan,magenta,orange,olive,pink,brown,grey
  `'i'` stands for `'iris'`: black,blue,cyan,green,yellow,orange,red,magenta,purple,grey

- **Choice of line style**
  Command: `setLine_style('style')`
  where `'style'` can be:
  - `cycle_colour` so that the line style changes only after a whole colour sequence;
  - `cycle_all` so that each curve is drawn with a different style;
  - any of the following: `'-', '- -', ':', '-.'`

- **Choice of point style**
  Command: `setPoint_style('style')`
  where `'style'` can be:
  - `cycle_colour` so that the points style changes only after a whole colour sequence;
  - `cycle_all` so that each model points are drawn with a different style;
  - any of the following: `'o','^','*','s','p','v','d','>','<'`

5.3 Windows

- **Drawing of 2 or 4 plots on the same window**
  Command: `multiPlot(2 or 4)`
  NB: back to normal with `multiPlot(1)`

5.4 Navigation within the graphs

The python window allows to easily zoom on a part of the graph after a clic on . It is also possible to navigate within the graph after clicking on . The original view is recovered with the button.

- **Activation of the cursor**
  Command: `cursor()`
  The coordinates of clicks are written on the terminal; to exit the cursor mode, just hit `Enter` on the terminal window.

- **Measurement of the distance between two points**
  Command: `dist()`
  The clics are validated by hitting `Enter` on the terminal window. The coordinates are written there, as well as Δx, Δy and the distance.

5.5 Interventions on the graphs

- **Superimposing plots**
  Command: `plot2var(mode)`
  By default, the mode is `'same'` that draws all the variables on the same y axis. The mode mode `'double'` allows to draw the 2nd variable with an independent axis on the right.
  NB: back to normal with `plot1var()`

- **Drawing a vertical line at the x value or a horizontal line at the y value**
  Command: `xline(x_value), yline(y_value)`

- **Drawing a line from(x_1,y_1) to (x_2,y_2)**
  Command: `line(`x_1`,`x_2`,`y_1`,`y_2`)`

- **Drawing a dot at the coordinate (x,y)**
  Command: `dotxy(x,y)`

- **Addition of external data on a graph**
  Command: `plotExternal('#File_name', x_column,y_column)`
  The following optional arguments are accepted:
  - `skip=...` to skip header lines in the file `#File_name` ;
  - `style=...` to define the points/lines style (example: `'ro','b+','g.'`) ;
  - `log='x','y' or 'xy'` to take the log of the variable(s) ;
  - `new` to create a new figure rather than adding the data to the current figure.

- **Addition of text in the graph**
  Command: `x,y,'string'`
  The optional argument `fontsize=...` modifies the fontsize (default: 24)

- **Addition of a title at the top of the window**
  Command: `top_label('string')`
  The optional argument `fontsize=...` modifies the fontsize (default: 24)

6 List of the variables

6.1 In EVOL mode

- MODEL:

line :	model num	t_tauH :	$t/\tau_{\rm H}$
M :	M $[M_\odot]$	ageadv :	$\log$(time before collapse [yr])
t :	t [yr]	tauKH :	τ_{KH} [yr]
t6 :	t [Myr]	rhom :	$\rho_{\rm m}$ [g cm^3]
t9 :	t [Gyr]		

Besides, the following global variables are known:

FileName :	`#loaded_file`
Mini :	$M_{\rm ini}$ $[M_\odot]$
tau :	lifetimes [yr] ([$\tau_{\rm H}$,$\tau_{\rm He}$,$\tau_{\rm C}$,$\tau_{\rm Ne}$,$\tau_{\rm O}$])
format :	`#format`

- CENTRE:

Mcc :	$M_{\rm cc}$ $[M_\odot]$
Mccrel :	$M_{\rm cc}/M_{\rm tot}$
rhoc :	$\log(\rho_{\rm c}$ [g cm^{-3}])
Tc :	$\log(T_{\rm c}$ $[K]$)

- SURFACE:

Teff :	$\log(T_{\rm eff}$ [K])	R :	R $[R_\odot]$
Teffncorr :	$\log(T_{\rm eff}$ [K]) not corrected for the wind thickness (WR)	Rpol :	$R_{\rm pol}$ $[R_\odot]$
L :	$\log(L/L_\odot)$	gsurf :	$\log(g_{\rm surf}$ [cm s^{-2}])
Mbol :	$\rm M_{bol}$	gpol :	$\log(g_{\rm pol}$ [cm s^{-2}])
GammaEdd :	$\Gamma_{\rm Edd}$	ZCext :	$M_{\rm ZC,ext}$

- ROTATION:

Omega_surf :	$\Omega_{\rm surf}$ [s^{-1}]	GammaOmega :	$\Omega/min(\Omega_{\rm crit,1}, \Omega_{\rm crit,2})$
Omega_cen :	$\Omega_{\rm cen}$ [s^{-1}]	rot_corr :	F_Ω
OOc :	$\Omega/\Omega_{\rm crit}$	Ltot :	$\mathcal{L}_{\rm tot}$ [10^{53} g cm^2 s^{-1}]
Vsurf :	$V_{\rm surf}$ [km s^{-1}]	Ltotint :	$\mathcal{L}_{\rm tot,int}$ [10^{53} g cm^2 s^{-1}]
Vcrit1 :	$V_{\rm crit,1}$ [km s^{-1}]	jspe5 :	$j_{5M_\odot}$ [10^{16} cm^2 s^{-1}]
Vcrit2 :	$V_{\rm crit,2}$ [km s^{-1}]	jspe3 :	$j_{3M_\odot}$ [10^{16} cm^2 s^{-1}]
oblat :	$R_{\rm pol}/R_{\rm eq}$	mominert :	I [10^{57} g cm^2]

- WINDS:

Mdot :	$\log(\dot{M}$ [$M_\odot$ yr^{-1}])	Vesc :	$V_{\rm esc}$ [km s^{-1}]
Mdot_mech :	$(\log(\dot{M})_{\rm mech}$ [$M_\odot$ yr^{-1}])	dlelex :	$\Delta\,\mathcal{L}_{\rm rad+aniso+mech}$ [10^{53} g cm^2 s^{-1}]
dMmech :	$\mathrm{d}M_{\rm mech}$ $[M_\odot]$	Bmin :	$B_{\rm min}$ [G] (minimal magn. field for a wind-surface coupling)
Pwinds :	$P_{\rm winds}$ [erg s^{-1}]		
Vinf :	V_∞ [km s^{-1}]		

- ENERGETICS:

Epot :	$E_{\rm pot}$ $[E_{51}]$
Egaz :	$E_{\rm th,gaz}$ $[E_{51}]$
Erad :	$E_{\rm rad}$ $[E_{51}]$
Erot :	$E_{\rm rot}$ $[E_{51}]$

- ABUNDANCES:

H1s,H1c :	^{1}H (surf., centr.) [mass frac.]	Be7c :	^{7}Be (centr.) [mass frac.]
He3s,He3c :	^{3}He (surf., centr.) [mass frac.]	B8c :	^{8}B (centr.) [mass frac.]
He4s,He4c :	^{4}He (surf., centr.) [mass frac.]	C12s,C12c :	^{12}C (surf., centr.) [mass frac.]

C13s,C13c :	^{13}C (surf., centr.) [mass frac.]	C12C13 :	$\log(^{12}\mathrm{C}/^{13}\mathrm{C}$ [numb.])
N14s,N14c :	^{14}N (surf., centr.) [mass frac.]	C12C13rel :	$\log(^{12}\mathrm{C}/^{13}\mathrm{C}) - \log(^{12}\mathrm{C}/^{13}\mathrm{C})_{\mathrm{ini}}$
O16s,O16c :	^{16}O (surf., centr.) [mass frac.]	NH :	$\log(\mathrm{N/H}$ [numb.] $+ 12)$
O17s,O17c :	^{17}O (surf., centr.) [mass frac.]	NHrel :	$\log(\mathrm{N/H}) - \log(\mathrm{N/H})_{\mathrm{ini}}$
O18s,O18c :	^{18}O (surf., centr.) [mass frac.]	NC :	$\log(\mathrm{N/C}$ [numb.])
Ne20s,Ne20c :	^{20}Ne (surf., centr.) [mass frac.]	NCrel :	$\log(\mathrm{N/C}) - \log(\mathrm{N/C})_{\mathrm{ini}}$
Ne22s,Ne22c :	^{22}Ne (surf., centr.) [mass frac.]	NO :	$\log(\mathrm{N/O}$ [numb.])
Al26s,Al26c :	^{26}Al (surf., centr.) [mass frac.]	NOrel :	$\log(\mathrm{N/O}) - \log(\mathrm{N/O})_{\mathrm{ini}}$
Zsurf :	Z_{surf} [mass frac.]		

6.2 In STRUC mode

- STRUCTURE:

shell :	shell number	rprev :	r_{prev} $[R_\odot]$
Mfrac :	M_r/M_{tot}	g :	g_r [cm s^{-2}]
Mr :	M_r $[M_\odot]$	N2 :	N^2 [s^{-1}]
r_cm :	r [cm]	Nmu2 :	N_μ^2 [s^{-1}]
r :	r $[R_\odot]$	NT2 :	N_T^2 [s^{-1}]

Besides, the following global variables are known:

M_tot :	M_{tot} $[M_\odot]$
FileName :	`#loaded_file`
Model :	model number
nshell :	total shells number
age :	age [yr]
timestep :	$\delta\, t$ [s]

- ENERGY:

L :	L_r/L_{tot}	epsOagNe :	$\epsilon(^{16}O(\alpha,\gamma)^{20}Ne)$ [erg g^{-1}s^{-1}]
epsH :	ϵ(H) [erg g^{-1}s^{-1}]	epsnu :	$-\epsilon_\nu$ [erg g^{-1}s^{-1}]
epsHe :	ϵ(He) [erg g^{-1}s^{-1}]	eps_reac :	$\epsilon_{\mathrm{nucl}} + \epsilon_\nu$ [erg g^{-1}s^{-1}]
epsC :	ϵ(C) [erg g^{-1}s^{-1}]	epsgrav :	ϵ_{grav} [erg g^{-1}s^{-1}]
eps3a :	$\epsilon(3\alpha)$ [erg g^{-1}s^{-1}]	dEdP :	$\mathrm{d}\ln E/\mathrm{d}\ln P$
epsCagO :	$\epsilon(^{12}C(\alpha,\gamma)^{16}O)$ [erg g^{-1}s^{-1}]	dEdT :	$\mathrm{d}\ln E/\mathrm{d}\ln T$

- EOS:

rho :	ρ [g cm^3]	Nabmu :	∇_μ
mu :	μ	drhodP :	$\mathrm{d}\ln\rho/\mathrm{d}\ln P$
muprev :	μ_{prev}	delta :	$\delta = -\mathrm{d}\ln\rho/\mathrm{d}\ln T$
mufit :	μ_{fit}	cs :	c_{sound} [cm s^{-1}]
mue :	μ_e	psi :	ψ (indicator of degeneracy)

- THERMO:

P :	P [g cm^{-1} s^{-2}]	Nabrad :	∇_{rad}
beta :	$\beta = P_{\mathrm{gas}}/P_{\mathrm{tot}}$	Nabla_int :	∇_{int}
Hp :	H_P [cm]	kappa :	κ [cm^2 g^{-1}]
T :	T $[K]$	dkdP :	$\mathrm{d}\ln\kappa/\mathrm{d}\ln P$
Kther :	K_{ther} [cm^2 s^{-1}]	dkdT :	$\mathrm{d}\ln\kappa/\mathrm{d}\ln T$
Nabla :	∇	Cp :	C_P [ergs g^{-1} K^{-1}]
Nabad :	∇_{ad}	V_MLT :	V_{MLT} [cm s^{-1}]

- ROTATION:

Omega :	Ω [s^{-1}]	Vr :	V_r [cm s^{-1}]
Omegaprev :	Ω_{prev} [s^{-1}]	Veq :	V_{eq} [km s^{-1}]
Omegacons :	$\Omega_{\mathcal{L},\mathrm{cons.}}$ [s^{-1}]	Lang :	$\mathcal{L}_r$ [g cm^2 s^{-1}]
Omfit :	Ω_{fit} [s^{-1}]	jr :	j_r [cm^{2}s^{-1}]
dlodlr :	$\mathrm{d}\ln\Omega/\mathrm{d}\ln r$	jS :	$j_{\mathrm{Schwarzschild}}$ [cm^{2}s^{-1}]
obla :	$r_{\mathrm{pol}}/r_{\mathrm{eq}}$	jK :	j_{Kerr} [cm^{2}s^{-1}]

jKmax :	$j_{\rm Kerr}^{\rm max}$ [cm^{2}s^{-1}]	Dconv :	$D_{\rm conv}$ [cm^2 s^{-1}]
Dh :	$D_{\rm h}$ [cm^2 s^{-1}]	Ur :	U_r [cm s^{-1}]
Dshear :	$D_{\rm shear}$ [cm^2 s^{-1}]	Richardson :	Ri $= N^2/({\rm d}V/{\rm d}z)^2$
Deff :	$D_{\rm eff}$ [cm^2 s^{-1}]		
Dcirc :	$D_{\rm circ}$ [cm^2 s^{-1}]		

• MAGNETISM:

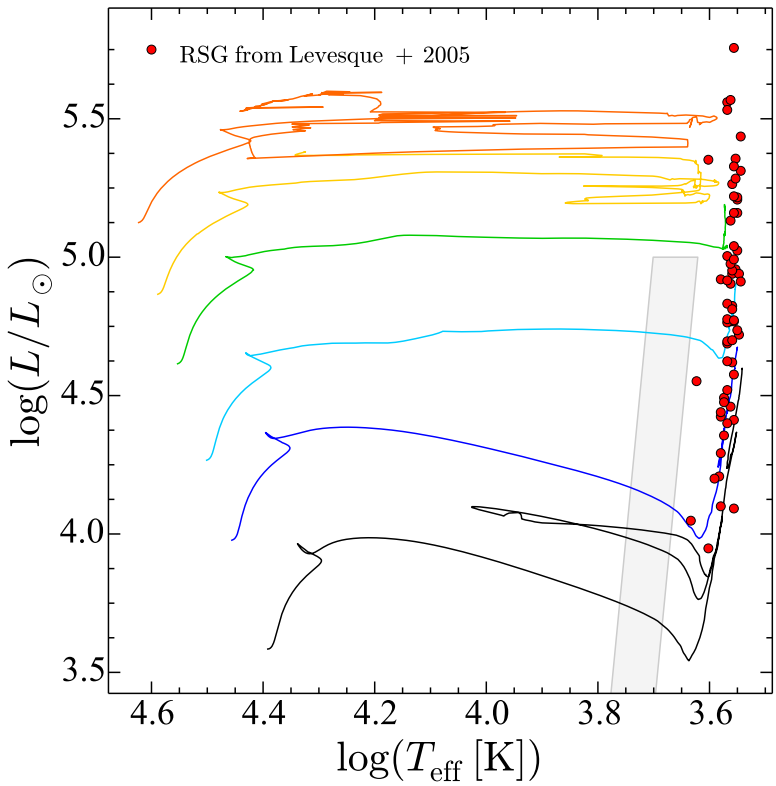
Br :	B_r $[G]$	etask :	η/K
Bphi :	B_ϕ $[G]$	N2mag :	$N^2_{\rm mag}$ [s^{-1}]
qmin :	$q_{\rm min}$	DmagO :	$D_{{\rm mag},\Omega}$ [cm^2 s^{-1}]
alfven :	$\omega_{\rm Alfven}$ [s^{-1}]	DmagX :	$D_{{\rm mag},X}$ [cm^2 s^{-1}]

• ABUNDANCES:

H1 :	^{1}H [mass frac.]	Si28,Si28_alu :	^{28}Si [mass frac.]
He3,He4 :	^{3,4}He [mass frac.]	S32 :	^{32}S [mass frac.]
C12,C13,C14 :	^{12,13,14}C [mass frac.]	Ar36 :	^{36}Ar [mass frac.]
N14,N15 :	^{14,15}N [mass frac.]	Ca40 :	^{40}Ca [mass frac.]
O16,O17,O18 :	^{16,17,18}O [mass frac.]	Ti44 :	^{44}Ti [mass frac.]
F18,F19 :	^{18,19}F [mass frac.]	Cr48 :	^{48}Cr [mass frac.]
Ne20,Ne21,Ne22 :	^{20,21,22}Ne [mass frac.]	Fe52 :	^{52}Fe [mass frac.]
Na23 :	^{23}Na [mass frac.]	Ni56 :	^{56}Ni [mass frac.]
Mg24,Mg25,Mg26 :	^{24,25,26}Mg [mass frac.]	protons :	protons [mass frac.]
Al26,Al27 :	^{26,27}Al [mass frac.]	neutrons :	neutrons [mass frac.]

7 Examples

7.1 HR diagram with observational data points



Loading of the package:

```
from origin_tools.Origin_Tools import *
```

Loading of the evolution files:

```
loadE('~/OBS/Grids2010/wgred/Z014/P009z14S0.dat',1)
loadE('~/OBS/Grids2010/wgred/Z014/P012z14S0.dat',2)
loadE('~/OBS/Grids2010/wgred/Z014/P015z14S0.dat',3)
loadE('~/OBS/Grids2010/wgred/Z014/P020z14S0.dat',4)
loadE('~/OBS/Grids2010/wgred/Z014/P025z14S0.dat',5)
loadE('~/OBS/Grids2010/tables/Z014/M032Z14V0.dat',6,format='tgrids')
```

Choice of the colour sequence `'iris'`:

```
set_colourSequence('i')
```

Change of the upper limit in y:

```
Limits(ymax=5.9)
```

Drawing of the HR diagram (predefined command):

```
HRD_plot()
```

Passing to `'points'` mode:

```
Points(True)
```

Drawing of external data, read in a file:

```
plotExternal('RSG_Levesque2005.dat',0,2,log='x',style='ro')
```

Drawing of a dot for the legend:

```
dotxy(4.6,5.75,style='ro')
```

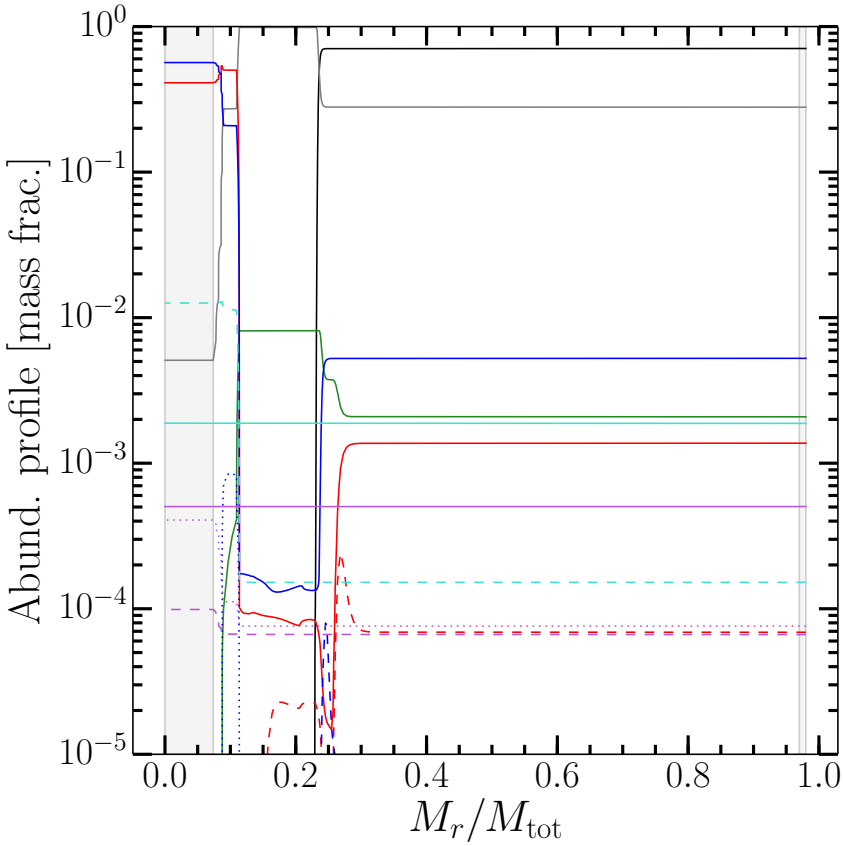
Addition of the legend text:

```
add_label(4.55,5.7,'$\mathrm{RSG\ from\ Levesque+2005}$',fontsize=16)
```

Saving the figure:

```
MyFig('HRDrainbow_RSG')
```

7.2 Abundances profiles with the convective zones shaded



Loading of a structure file:

```
loadS('~/OBS/Calc/testVrac/P007z14S0.v0022721',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Enhancement of the axes ticks:

```
ChangeTickSize(length=12,width=2)
```

Modification of the lower limit in y:

```
Limits(ymin=1.e-5)
```

y-axis in logarithmic scale:

```
logScale('y')
```

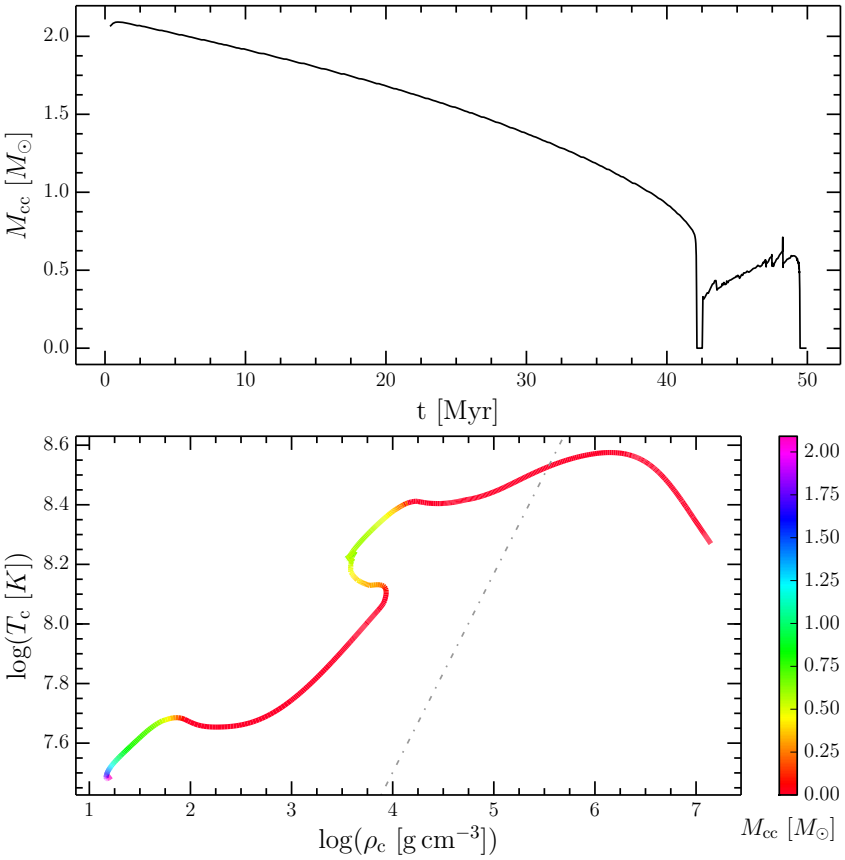
Drawing of the abundances profiles (predefined command):

```
Abund('p')
```

Addition of the convective zones as shaded areas:

```
convZones(1)
```

7.3 Double figure with one of them having a 3rd variable in colour code



Loading of an evolution file:

```
loadE('/Users/ekstrom/OBS/Grids2010/wgred/Z014/P007z14S0.dat',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Passing to two figures in a window mode:

```
multiPlot(2)
```

Reduction of the labels size:

```
ChangeFontSize(12)
```

Drawing of the firrst figure (default x-axis):

```
Plot('Mcc')
```

Choice of the x-axis for the 2nd figure:

```
defX('rhoc')
```

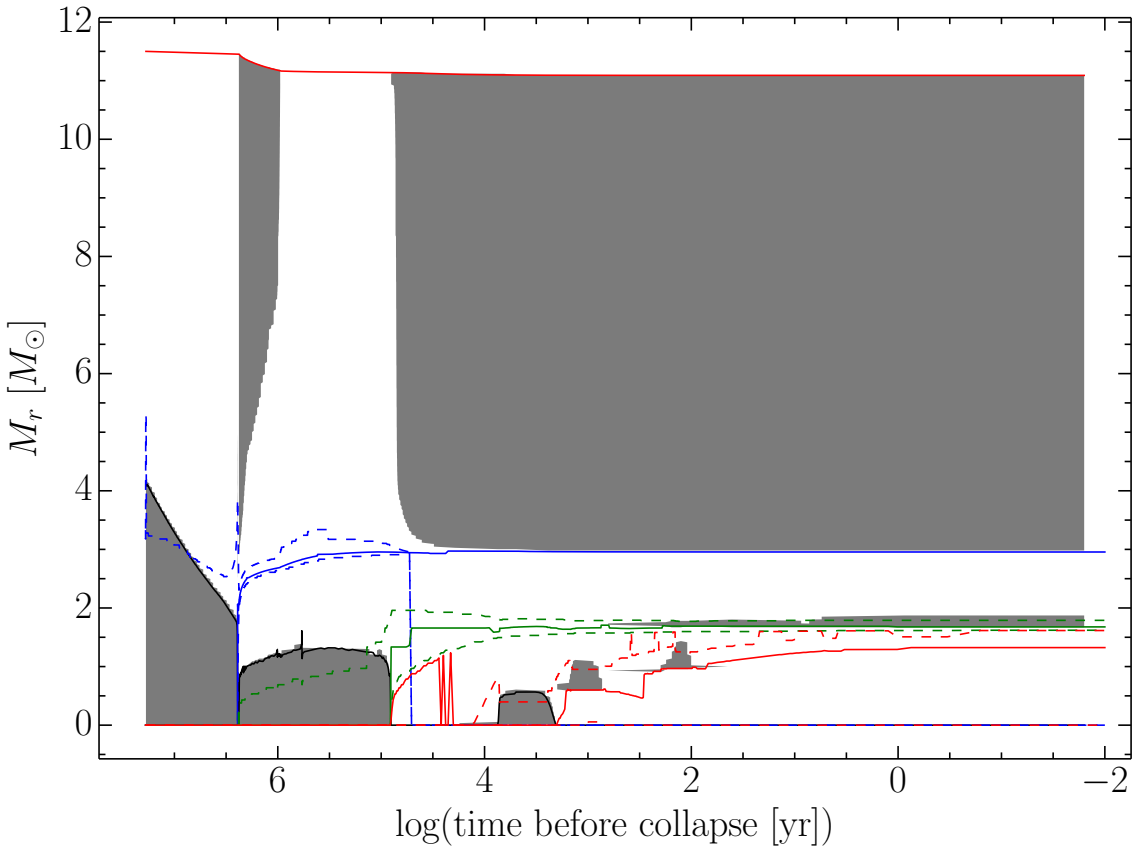
Drawing with a 3rd variable (`Mcc`) in colour code:

```
Plot_colour('Tc','Mcc')
```

Addition of the gas degeneracy line on the plot:

```
degenerate_line()
```

7.4 Kippenhahn diagram with burning zones



Loading of an evolution file:

```
loadE('/Users/ekstrom/OBS/Grids2010/wgred/Z014/P011p5z14S0.dat',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Definition of the x variable:

```
defX('ageadv')
```

Drawing of the Kippenhahn diagram (predefined command):

```
Kippen(1,burn=True)
```

NB : the burning zones (plotted with the optional argument `burn=True`) are read in a `#starname.burn` file that must exist somewhere. This `.burn` file is automatically generated when the star's directory has been cleaned with the `cleanfiles` routine with the option `-b`. it can be generated afterwards on the `.v` file of a model with the script `BurningZonesCalc.bash` that runs on the `strucData` program. All those programs and routines can be found on the `svn`.